CGE-P LAB CURRICULUM · V2

Lab 5.4: GCP Security Services Baseline

GCP leads with identity and data. Where AWS gives you a flat IAM policy and a Security Hub aggregator, GCP gives you Org Policy that rejects misconfigurations at the API call, Workload Identity Federation that replaces service account keys with short-lived OIDC tokens, and Data Access logs that, deliberately, you have to turn on.

This lab does all three for one project, including the part most guidance skips: a safe way to introduce a preventive control without breaking production on the day you enable it.

For reading. Code lines wrap to fit the page; copy code from the repository, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from the repository (`GRCEngClub/cgep-labs`), not from the PDF.

Learning objectives

- Enforce Org Policy at the API so non-compliant creation is rejected before the resource exists.
- **Roll out a preventive control in audit mode first**, then enforce. This is new in v2.
- Replace long-lived service account keys with Workload Identity Federation.
- Enable Data Access audit logs, the single most-cited GCP audit finding.

Controls implemented

Control	Mechanism
CM-6	`storage.uniformBucketLevelAccess`, `storage.publicAccessPrevention`
AC-2, IA-5	`iam.disableServiceAccountKeyCreation` + WIF
AC-3	`compute.requireOsLogin`
AU-2, AU-3, AU-12	Data Access audit logs per service
SC-7	VPC Service Controls (discussed, not deployed; see Step 6)
SC-8	**Inherited from GCP.** Cloud Storage and the Google APIs are TLS-only.

Same inherited-control note as Lab 2.4. On AWS you write a bucket policy to get SC-8; on GCP the platform provides it. Record it as `inherited` in your OSCAL rather than claiming you implemented it.

Prerequisites

- **Lab 0.1 Part 2** for project, billing, APIs and both authentications. `orgpolicy` and `sts` are the two APIs people forget, and Step 8 enables them.
- A GCP project you own with billing enabled, and APIs `cloudkms`, `iam`, `cloudresourcemanager`, and `orgpolicy` enabled.
- Roles: `roles/orgpolicy.policyAdmin`, `roles/iam.workloadIdentityPoolAdmin`, `roles/logging.admin`. At project scope your project owner can grant these; at org scope you need org-level rights.
- `gcloud auth login` **and** `gcloud auth application-default login`.
- Terraform `>= 1.10`.

The lab uses project scope so it works in any environment. If your project sits in an Organization and you want org or folder scope, the same resources take a different `parent`.

Estimated time & cost

- 90 minutes, mostly waiting for Org Policy propagation (5 to 10 minutes per change).
- Org Policy: free. WIF: free. Security Command Center Standard: free at org level.
- **Data Access logs: $0.50/GB ingested plus Cloud Logging storage.** Pennies for an empty project, gigabytes per day in a busy one. Start with one service.

Architecture

```
project (your-gcp-project)
--------------------------
Org Policy (project scope, REJECT at the API):
  storage.uniformBucketLevelAccess       = TRUE     CM-6
  storage.publicAccessPrevention         = TRUE     AC-3
  iam.disableServiceAccountKeyCreation   = TRUE     AC-2 / IA-5
  compute.requireOsLogin                 = TRUE     AC-3

Workload Identity Federation:
  pool     : github-actions
  provider : token.actions.githubusercontent.com
  condition: assertion.repository == "GRCEngClub/cgep-app-starter"
  SA       : cgep-grc-gate-sa@...  (roles/viewer)

Data Access audit logs (per service):        AU-2 / AU-3 / AU-12
  storage.googleapis.com    DATA_READ + DATA_WRITE + ADMIN_READ
  cloudkms.googleapis.com   DATA_READ + DATA_WRITE + ADMIN_READ
  iam.googleapis.com        DATA_READ + DATA_WRITE + ADMIN_READ
```

Step-by-step walkthrough

Concept: why identity-first

GCP's bet is that the smallest unit of security is the principal, not the resource.

Org Policy enforces at the API call: a bucket creation that violates `uniformBucketLevelAccess` is **rejected**, not flagged. WIF replaces "create a service account, download a JSON key, paste it into GitHub Secrets, hope nobody leaks it" with "the runtime presents an OIDC token, GCP swaps it for a short-lived access token, the token expires by itself."

Together they make whole categories of attack uneconomical. Compare the layers you now have:

Layer	Example	When it acts
Preventive, at the API	Org Policy	The action never happens
Preventive, at the pipeline	Conftest gate (Ch 3)	Before merge
Detective	Security Hub, SCC, Config	Minutes to hours after

Org Policy is the strongest layer, and it is the one that will page you at 2am if you enable it carelessly. Hence Step 2.

Step 1 Org Policy in audit mode first

This is the v2 addition and the most useful habit in the lab.

Going straight to `enforce = "TRUE"` is fine in a lab account. In a real project, turning on a rejection at the API without knowing what currently violates it is how you break the deploy pipeline of a team that has never heard of you.

The Org Policy v2 API supports a dry-run specification: the constraint evaluates and logs violations without rejecting anything.

```
# Phase 1: observe. Violations are logged, nothing is blocked.
resource "google_org_policy_policy" "uniform_bucket_access" {
  name   = "projects/${var.gcp_project}/policies/storage.uniformBucketLevelAccess"
  parent = "projects/${var.gcp_project}"

  dry_run_spec {
    rules {
      enforce = "TRUE"
    }
  }
}
```

Leave it for a week. Query the violations:

```
gcloud logging read \
  'protoPayload.status.details.violations.type="ORG_POLICY_DRY_RUN"' \
  --project=your-gcp-project --limit=50 --format=json
```

Then, once the list is empty or every entry is understood, promote it:

```
# Phase 2: enforce.
resource "google_org_policy_policy" "uniform_bucket_access" {
  name   = "projects/${var.gcp_project}/policies/storage.uniformBucketLevelAccess"
  parent = "projects/${var.gcp_project}"

  spec {
```

```
    rules {
      enforce = "TRUE"
    }
  }
}
```

Do not audit by omitting the rules block. It is a natural guess and it is dangerously wrong: a policy with no rules is not auditing, it is **not in effect at all**. You would observe zero violations and conclude you were compliant. `dry_run_spec` is the mechanism that actually audits. Confirm your `google` provider supports it; it is present in recent 5.x.

The rest of the constraints, in their enforcing form:

```
resource "google_org_policy_policy" "public_access_prevention" {
  name   = "projects/${var.gcp_project}/policies/storage.publicAccessPrevention"
  parent = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}

resource "google_org_policy_policy" "disable_sa_keys" {
  name   = "projects/${var.gcp_project}/policies/iam.disableServiceAccountKeyCreation"
  parent = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}

resource "google_org_policy_policy" "require_oslogin" {
  name   = "projects/${var.gcp_project}/policies/compute.requireOsLogin"
  parent = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}
```

`storage.publicAccessPrevention` is new in v2 and pairs with Lab 2.4's per-bucket setting. The module sets it on buckets it creates; the org policy sets it on buckets **anyone** creates, including by hand in the console. That is the difference between a module standard and an organizational control, and it is worth saying out loud in your write-up.

Step 1b Apply it

Nothing above has reached GCP yet. Step 2 tests constraints that do not exist until you apply, and skipping this is easy because Step 1 ends in a wall of HCL rather than a command.

`gcp_project`, `github_org` and `github_repo` have no defaults, because a default project is precisely how you enforce an org policy on the wrong project. Put them in `terraform.tfvars`, which Terraform reads automatically and which is gitignored:

```
cat > terraform.tfvars <<'EOF'
gcp_project = "your-gcp-project"
github_org  = "YourOrg"
github_repo = "YourRepo"
EOF
```

```
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Apply the **enforcing** form before testing. If `uniform_bucket_access` is still on `dry_run_spec`, Step 2's bucket test will succeed, and it will look like a broken control rather than a policy that is deliberately only watching.

Without the tfvars file Terraform prompts for all three values on every `plan`, `apply` and `destroy`, and fails outright under `-input=false`.

Step 2 Test the enforcement

```
gcloud iam service-accounts keys create /tmp/key.json \
  --iam-account=YOUR_SA_EMAIL --project=your-gcp-project
```

```
ERROR: (gcloud.iam.service-accounts.keys.create) FAILED_PRECONDITION:
Key creation is not allowed on this service account.
constraint iam.disableServiceAccountKeyCreation
```

This is the lesson. The control did not fire after the fact in a finding three hours later. **The action did not happen.** Capture that transcript; it is better evidence than any configuration dump, because it demonstrates enforcement rather than intent.

Do the same for the bucket constraint:

```
gcloud storage buckets create gs://your-project-nonuniform-test \
  --project=your-gcp-project --no-uniform-bucket-level-access
# expect: rejected by constraint storage.uniformBucketLevelAccess
```

Step 3 Workload Identity Federation

The `attribute_condition` is the whole security of this construct. Without it, **any GitHub repository on the public internet** can present a token to your provider and impersonate your service account.

```
resource "google_iam_workload_identity_pool" "github" {
  workload_identity_pool_id = "github-actions"
  display_name              = "GitHub Actions"
}

resource "google_iam_workload_identity_pool_provider" "github" {
  workload_identity_pool_id          =
google_iam_workload_identity_pool.github.workload_identity_pool_id
  workload_identity_pool_provider_id = "github"

  attribute_mapping = {
    "google.subject"       = "assertion.sub"
    "attribute.repository" = "assertion.repository"
    "attribute.actor"      = "assertion.actor"
    "attribute.ref"        = "assertion.ref"
  }

  # AC-3. Without this line the provider trusts all of GitHub.
  attribute_condition = "assertion.repository == \"${var.github_org}/${var.github_repo}\""

  oidc {
    issuer_uri = "https://token.actions.githubusercontent.com"
  }
}

resource "google_service_account" "gha" {
  account_id   = "cgep-grc-gate-sa"
  display_name = "CGE-P GRC gate (read-only)"
}

# AC-6: read-only. The gate plans and inspects; it does not change anything.
resource "google_project_iam_member" "gha_viewer" {
  project = var.gcp_project
  role    = "roles/viewer"
  member  = "serviceAccount:${google_service_account.gha.email}"
}

resource "google_service_account_iam_binding" "wif_user" {
  service_account_id = google_service_account.gha.name
  role               = "roles/iam.workloadIdentityUser"

  members = [
    "principalSet://iam.googleapis.com/${google_iam_workload_identity_pool.github.name}/
attribute.repository/${var.github_org}/${var.github_repo}",
  ]
}
```

Defense in depth here is two layers, and students routinely build only one. The `attribute_condition` on the provider decides who may exchange a token at all. The `principalSet` in the binding decides who may impersonate this particular service account. Set the condition and use `principalSet://.../*` in the binding and you have one layer. Set both and a mistake in either is survivable.

For an apply pipeline, tighten further to a specific ref:

```
principalSet://iam.googleapis.com/POOL/attribute.ref/refs/heads/main
```

The workflow side, keyless:

```
permissions:
  id-token: write
  contents: read

steps:
  - uses: google-github-actions/auth@7c6bc770dae815cd3e89ee6cdf493a5fab2cc093 # v3
    with:
      workload_identity_provider: projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/
github-actions/providers/github
      service_account: cgep-grc-gate-sa@your-gcp-project.iam.gserviceaccount.com

  - run: gcloud storage ls
```

The token is minted at job start, expires in an hour, and never touches disk. Same posture as the AWS OIDC pattern in Lab 4.3, different cloud, identical argument. Note the action is SHA-pinned for the same supply chain reason.

Step 4 Data Access audit logs

Off by default, and the most-cited GCP audit finding because nobody turns them on.

```
resource "google_project_iam_audit_config" "storage" {
  project = var.gcp_project
  service = "storage.googleapis.com"
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
  audit_log_config { log_type = "ADMIN_READ" }
}

resource "google_project_iam_audit_config" "kms" {
  project = var.gcp_project
  service = "cloudkms.googleapis.com"
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
  audit_log_config { log_type = "ADMIN_READ" }
}

resource "google_project_iam_audit_config" "iam" {
  project = var.gcp_project
  service = "iam.googleapis.com"
  audit_log_config { log_type = "ADMIN_READ" }
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
}
```

Verify a read actually lands:

```
gcloud storage ls gs://your-test-bucket
sleep 30
gcloud logging read \
  'protoPayload.serviceName="storage.googleapis.com" AND
protoPayload.methodName=~"storage.objects.list"' \
  --limit 5 --format=json
```

Turning them on is AU-2 and AU-12. Reading them is AU-6, and the same caution from Lab 5.2 applies: a log nobody reviews is a cost centre, not a control. Log Analytics or a BigQuery sink is the GCP equivalent of the Athena setup in Lab 5.2. Build one or claim `partial`.

Step 5 Security Command Center

If your project is in an Organization with SCC enabled, findings flow in automatically. Standard is free at org level; Premium is enterprise-priced and not used here. The lab does not provision SCC because it needs org admin.

```
gcloud scc findings list ORG_ID --source=- --format=json \
  > evidence/lab-5-4/scc-findings.json
```

For a standalone project without an Organization, SCC is unavailable. The Org Policy enforcements above are your preventive layer, and stating that substitution explicitly in your write-up is the right move: "detective coverage is absent at project scope; compensated by preventive enforcement at the API."

Step 6 What is missing, and naming it

Two real controls this lab does not build, worth knowing so you do not overclaim:

VPC Service Controls (SC-7). The genuine data-exfiltration boundary on GCP. A service perimeter stops a principal with valid credentials from copying data to a project outside the perimeter, which IAM alone cannot do. It requires org-level rights and careful planning, and misconfiguring one locks you out of your own project. Name it as a gap with a remediation plan.

Assured Workloads / CMEK org constraints. `gcp.restrictNonCmekServices` forces CMEK across services rather than relying on each module to do the right thing. It is the organizational form of what Lab 2.4's module does per-bucket, and it is the same relationship as `storage.publicAccessPrevention` in Step 1.

Verification

```
gcloud org-policies list --project=your-gcp-project \
  | grep -E "uniformBucket|publicAccessPrevention|disableServiceAccount|requireOsLogin"

gcloud iam workload-identity-pools list --location=global --project=your-gcp-project
```

```
gcloud projects get-iam-policy your-gcp-project --format=json \
  | jq '.auditConfigs'

# Enforcement, watched rather than assumed
gcloud iam service-accounts keys create /tmp/k.json \
  --iam-account=cgep-grc-gate-sa@your-gcp-project.iam.gserviceaccount.com \
  --project=your-gcp-project
# expect: FAILED_PRECONDITION
```

Portfolio submission checklist

- [ ] `terraform/baselines/gcp/` committed.
- [ ] At least one workflow using WIF. **No service account JSON key anywhere in the repo or its history.**
- [ ] `evidence/lab-5-4/iam-policy.json` capturing the Data Access log configuration.
- [ ] `evidence/lab-5-4/enforcement-denied.txt`, the transcripts of both rejected actions.
- [ ] README notes the "Data Access logs are off by default" lesson, and names VPC Service Controls as a known gap.

Troubleshooting

- **Org Policy propagation latency.** First-apply changes take 5 to 10 minutes. A test run immediately after `terraform apply` may briefly succeed. That is propagation, not a broken policy.
- **`PERMISSION_DENIED` on the WIF provider.** You need `roles/iam.workloadIdentityPoolAdmin` specifically. Being project Owner is not sufficient.
- **`attribute.repository` mismatch.** GitHub's `assertion.repository` is the literal `OWNER/REPO`. Case, spelling, and the slash all matter. An opaque `PERMISSION_DENIED` from `auth@v2` is almost always this.
- **`policySpec is not supported`.** Enable the v2 API: `gcloud services enable orgpolicy.googleapis.com`.
- **`dry_run_spec` unrecognized.** Your `google` provider is too old. Upgrade, or accept that you are enforcing without an observation period and say so.
- **Data Access log cost.** In a busy project these ingest gigabytes per day. Start with `storage.googleapis.com` alone before adding KMS and IAM.

Cleanup

```
terraform destroy -auto-approve
```

Three things that will surprise you:

1. **WIF pools enter a 30-day soft delete.** You cannot recreate a pool with the same ID until it expires, or you `gcloud iam workload-identity-pools undelete` and then delete again with `--purge`.
2. **Disabling Org Policy does not retroactively un-enforce.** Buckets created with uniform access stay that way. The policy only stops blocking new violations.
3. **Audit config removal is not retroactive either.** Logs already ingested continue to bill for storage until their retention expires.

How this feeds the capstone

The WIF pattern is your AWS-OIDC equivalent for anything GCP-touching. If your capstone pipeline reaches into GCP for any reason, it uses WIF, never a key.

The Org Policy layer is the preventive tier above your Rego, which is detective. Saying that clearly in your write-up, with the three-layer table from the Concept section, is one of the strongest paragraphs you can write about defense in depth, because it shows you know which of your controls acts when.

The Data Access logs feed your OSCAL component's AU-2 and AU-3 statements: enabled per service, with the IAM policy JSON as evidence. And the audit-mode-then-enforce rollout in Step 1 is the answer to the write-up question "how would you introduce this control without breaking the engineering team," which the capstone brief asks in the form "without slowing the engineering team down."

Revision history

v2 (current)

- Org Policy constraints roll out in `dry_run_spec` first and promote to `spec`, so a preventive control can be introduced without breaking a team that has never heard of you.
- Corrected the audit-mode guidance: omitting the rules block does not audit, it leaves the policy not in effect at all.
- Added `storage.publicAccessPrevention` alongside the original three constraints.
- Workload Identity Federation documented as two layers, the provider `attribute_condition` and the binding `principalSet`, rather than one.
- Named VPC Service Controls and CMEK org constraints as known gaps rather than leaving them unmentioned.

v1

Initial release: three constraints, straight to enforce, WIF documented as a single layer.